# Introduction to Software Engineering

Week 1

Chanduni Gamage
Lecturer (Probationary)
Dept. of ICT
Faculty of Technology
University of Ruhuna

# What is Software?

*Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information and (3) documentation that describes the operation and use of the programs.*

# What is Software?

- ***Software is developed or engineered, it is not manufactured in the classical sense.***
- ***Software doesn't "wear out."***
- ***Although the industry is moving toward component-based construction, most software continues to be custom-built.***

# Software Applications

- System Software
- Application Software
- Engineering/Scientific Software
- Embedded Software
- Product-line Software
- Webapps (Web Applications)
- AI Software

# Software—New Categories

- Open world computing—pervasive, distributed computing
- Ubiquitous computing—wireless networks
- Net sourcing—the web as a computing engine
- Open source—”free” source code open to the computing community (a blessing, but also a potential curse!)
- Also …
  - Data mining
  - Grid computing
  - Cognitive machines
  - Software for nanotechnologies

# Legacy Software

## *Why must it change?*

- Software must be adapted to meet the needs of new computing environments or technology.
- Software must be enhanced to implement new business requirements.
- Software must be extended to make it interoperable with other more modern systems or databases.
- Software must be re-architected to make it viable within a network environment.

# Software Engineering

- Some realities:
  - *A concerted effort should be made to understand the problem before a software solution is developed*
  - *Design becomes a pivotal activity*
  - *Software should exhibit high quality*
  - *Software should be maintainable*
- The seminal definition:
  - *[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*

# Software Engineering

- The IEEE definition:
  - *Software Engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).*

# Software engineering

- Software engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the early stages of system specification through to maintaining the system after it has gone into use.
- Engineering discipline
  - Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
- All aspects of software production
  - Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

# Major Software Failures

- **Ariane 5** On June 4th, 1996
- Europe's newest un-manned satellite-launching rocket reused working software from its predecessor, the Ariane 4.
- Unfortunately, the Ariane 5's faster engines exploited a bug that was not found in previous models.
- **Thirty-six seconds** into its maiden launch the rocket's **engineers hit the self destruct button** following multiple computer failures.
- In essence, the software had tried to cram a 64-bit number into a 16-bit space.
- The resulting overflow conditions crashed both the primary and backup computers (which were both running the exact same software).
- The Ariane 5 had cost nearly $8 billion to develop, and was carrying a $500 million satellite payload when it exploded
- This delayed scientific research into workings of the Earth's magnetosphere for almost 4 years



# Major Software Failures

- **Patriot Missile Error**
  - In February of 1991, a U.S. Patriot missile defence system in Saudi Arabia, failed to detect an attack on an Army barracks.
  - A government report found that a software problem led to an “inaccurate tracking calculation that became worse the longer the system operated.”
  - On the day of the incident, the system had been operating for more than 100 hours, and the inaccuracy was serious enough to cause the system to look in the wrong place for the incoming missile.
  - The attack killed 28 American soldiers. Prior to the incident, Army officials had fixed the software to improve the Patriot system’s accuracy.
  - That modified software reached the base the day after the attack.

# Major Software Failures

- **Read more…**
  - **Bitcoin Hack, Mt. Gox**
  - **Heathrow Terminal 5 Opening**
  - **The Morris Worm etc…**

# Importance of Software Engineering

- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

# Software Process Activities

- Software specification, where customers and engineers define the software that is to be produced and the constraints on its operation.
- Software development, where the software is designed and programmed.
- Software validation, where the software is checked to ensure that it is what the customer requires.
- Software evolution, where the software is modified to reflect changing customer and market requirements.

# General issues that affect most software

- Heterogeneity
  - Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.
- Business and social change
  - Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.
- Security and trust
  - As software is intertwined with all aspects of our lives, it is essential that we can trust that software.

# Software engineering diversity

- There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.
- The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.

# Application types

- Stand-alone applications
  - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- Interactive transaction-based applications
  - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- Embedded control systems
  - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

# Application types

- Batch processing systems
  - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- Entertainment systems
  - These are systems that are primarily for personal use and which are intended to entertain the user.
- Systems for modelling and simulation
  - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

# Application types

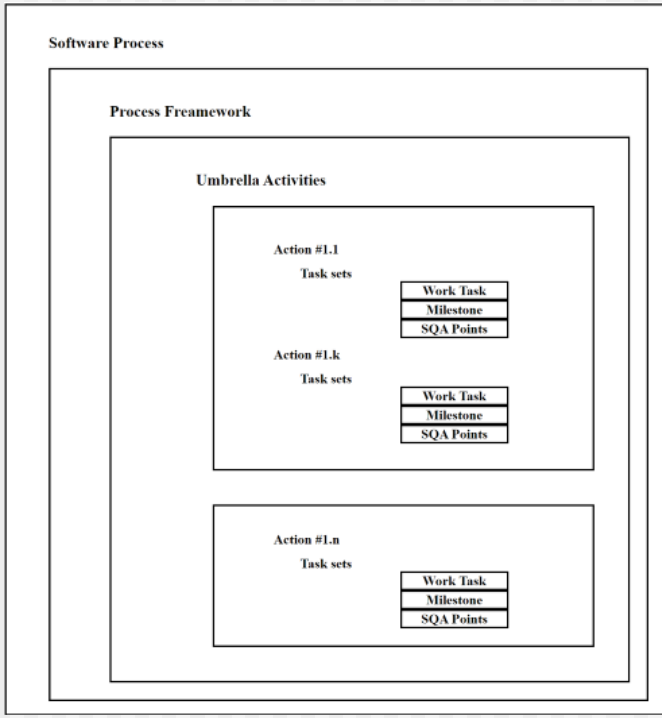
- Data collection systems
  - These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
- Systems of systems
  - These are systems that are composed of a number of other software systems.

# Software engineering fundamentals

- Some fundamental principles apply to all types of software system, irrespective of the development techniques used:
  - Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
  - Dependability and performance are important for all types of system.
  - Understanding and managing the software specification and requirements (what the software should do) are important.
  - Where appropriate, you should reuse software that has already been developed rather than write new software.

# Framework Activities

- Communication
- Planning
- Modeling
  - Analysis of requirements
  - Design
- Construction
  - Code generation
  - Testing
- Deployment



# Umbrella Activities

Umbrella Activities are that take place during a software development process for improved project management and tracking.

- Formal technical reviews
- Software quality assurance
- Software configuration management
- Work product preparation and production
- Reusability management
- Measurement
- Risk management

# Adapting a Process Model

- The overall flow of activities, actions, and tasks and the interdependencies among them
- The degree to which actions and tasks are defined within each framework activity
- The degree to which work products are identified and required
- The manner which quality assurance activities are applied
- The manner in which project tracking and control activities are applied
- The overall degree of detail and rigor with which the process is described
- The degree to which the customer and other stakeholders are involved with the project
- The level of autonomy given to the software team
- The degree to which team organization and roles are prescribed

Software Engineering: A Practitioner's Approach, 7/e (McGraw-Hill 2009). Slides copyright 2009 by Roger Pressman.

# The Essence of Practice

- Polya suggests:
  1. *Understand the problem* (communication and analysis).
  2. *Plan a solution* (modeling and software design).
  3. *Carry out the plan* (code generation).
  4. *Examine the result for accuracy* (testing and quality assurance).

# Understand the Problem

- *Who has a stake in the solution to the problem?* That is, who are the stakeholders?
- *What are the unknowns?* What data, functions, and features are required to properly solve the problem?
- *Can the problem be compartmentalized?* Is it possible to represent smaller problems that may be easier to understand?
- *Can the problem be represented graphically?* Can an analysis model be created?

# Plan the Solution

- *Have you seen similar problems before?* Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
- *Has a similar problem been solved?* If so, are elements of the solution reusable?
- *Can subproblems be defined?* If so, are solutions readily apparent for the subproblems?
- *Can you represent a solution in a manner that leads to effective implementation?* Can a design model be created?

# Carry Out the Plan

- *Does the solution conform to the plan?* Is source code traceable to the design model?
- *Is each component part of the solution provably correct?* Has the design and code been reviewed, or better, have correctness proofs been applied to algorithm?

# Examine the Result

- *Is it possible to test each component part of the solution?* Has a reasonable testing strategy been implemented?
- *Does the solution produce results that conform to the data, functions, and features that are required?* Has the software been validated against all stakeholder requirements?

# Software Myths

- Affect managers, customers (and other non-technical stakeholders) and practitioners
- Are believable because they often have elements of truth,

*but …*

- Invariably lead to bad decisions,

*therefore …*

- Insist on reality as you navigate your way through software engineering

# How It all Starts

- *SafeHome:*
  - Every software project is precipitated by some business need—
    - The need to correct a defect in an existing application;
    - The need to the need to adapt a ‘legacy system’ to a changing business environment;
    - The need to extend the functions and features of an existing application, or
    - The need to create a new product, service, or system.

# Thank You!